

Spring Security Configuration

Owen Lindsey

Professor Sluiter, Shad

CST-407

Grand Canyon University

Spring Security Coding activity

Download the **PREPACKAGED JAVA CODE UTILIZING SPRING** initialize the Maven repository, update and clean any package errors or paths. Then add Spring boot starter security.

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

This dependency is a collection of other security dependencies for authentication and authorization. The security dependencies included in the starter group include:

spring-security-core whose classes include:

Classes	Function	Typical Methods/Usage
AuthenticationManager	Orchestrates authentication by delegating to one or more providers	<code>authenticate(Authentication)</code>
UserDetails	Contract describing the authenticated user	Getters like <code>getUsername()</code> , <code>getPassword()</code> , authorities
UserService	Application service to manage users (domain + security integration)	<code>save(...)</code> , <code>findByLoginName(...)</code> , <code>loadUserByUsername(...)</code>
BCryptPasswordEncoder	Password hashing using BCrypt	<code>encode(rawPassword)</code> , <code>matches(raw, encoded)</code>
GrantedAuthority	Represents a permission/role granted to the user	Implementations like <code>SimpleGrantedAuthority("ROLE_ADMIN")</code>

spring-security-config whose classes include:

Class	Function
WebSecurityConfigurerAdapter	Legacy base class for configuring web security (deprecated in Spring Security 6)
HttpSecurity	Fluent API to configure web security: authorize rules, login, logout, CSRF, etc.
AuthenticationManagerBuilder	Builds the global <code>AuthenticationManager</code> (providers, <code>UserDetailsService</code> , encoders)

spring-security-web whose classes include:

Class	Function
UsernamePasswordAuthenticationFilter	Processes form login (<code>/login</code>) and attempts authentication
LogoutFilter	Handles logout requests and clears security context/session
BasicAuthenticationFilter	Processes HTTP Basic auth headers
RememberMeAuthenticationFilter	Restores authentication from remember-me cookie
CsrfFilter	Applies CSRF protection to state-changing HTTP requests
SecurityContextPersistenceFilter	Loads/saves <code>SecurityContext</code> for each request

Security Configuration Class

Now we will configure the Security class within our code. Start with creating a *config* folder in the root of *ordersapp*. Create a new class **SecurityConfig.java** in the *config* folder.

```
package com.shadsluiter.ordersapp.config;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final UserDetailsService userDetailsService;
    private final PasswordEncoder passwordEncoder;

    @Autowired
    public SecurityConfig(UserDetailsService userDetailsService, PasswordEncoder passwordEncoder) {
        this.userDetailsService = userDetailsService;
        this.passwordEncoder = passwordEncoder;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(authorizeHttpRequests ->
                authorizeHttpRequests
                    // allow anyone to access the registration and login pages
                    .requestMatchers("/users/register", "/users/loginForm", "/orders", "/", "/index", "/css/**", "/js/**").permitAll()
                    // only admin can access the edit and delete pages
                    .requestMatchers("/orders/edit/**", "/orders/delete/**").hasRole("ADMIN")
                    // all other requests must be authenticated. Home, Create.
                    .anyRequest().authenticated()
                )
            // define where the login and logout pages are located
            .formLogin(loginForm ->
                loginForm
                    .loginPage("/users/loginForm")
                    .loginProcessingUrl("/login")
                    .defaultSuccessUrl("/users/home", true) // Ensure this URL exists
                    .failureUrl("/users/loginForm?error=true")
                    .permitAll()
                )
            .logout(logout ->
                logout
                    .logoutUrl("/logout")
                    .logoutSuccessUrl("/index")
                    .permitAll()
                );

        return http.build();
    }

    // necessary because we are using a custom UserDetailsService and PasswordEncoder
    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
        auth.userDetailsService(userDetailsService).passwordEncoder(passwordEncoder);
    }
}
```

main

config

SecurityConfig.java

2, U

controllers

data

OrderRepository.java

M

OrderRepositoryInterface.j..

U

UserRepository.back

U

UserRepository.java

M

UserRepositoryInterface.java

U

models

service

OrdersappApplication.java

resources

test

target

concat.txt

U

HELP.md

U

mvnw

M

mvnw.cmd

U

ordersapproles.sql

U

pom.xml

M

OUTLINE

TIMELINE

JAVA PROJECTS

MAVEN

agms com.gcuagms

blockchaindemo.com blockchaindemo

the error seen is from the linter in the IDE nothing is wrong with this code.

****Password Encoding Config**

Create a new class **PasswordConfig.java** in the *config* folder. This code will use BCrypt hashing. See table for alternative hashing options.

```
package com.shadsluiter.ordersapp.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordEncoderConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

For Production: Use BCryptPasswordEncoder, Pbkdf2PasswordEncoder, SCryptPasswordEncoder, or Argon2PasswordEncoder.

For Testing: Use NoOpPasswordEncoder only in non-production environments for simplicity.

Algorithm	Security	Performance	Strengths	Weaknesses
BCryptPasswordEncoder	High	Medium	Resistant to rainbow table attacks, adjustable work factor	Slower than simple hashes like MD5
NoOpPasswordEncoder	None	Very High	Fastest, useful only for testing	Not secure at all, should not be used
Pbkdf2PasswordEncoder	High	Medium-High	Industry standard, configurable key length and iterations	Computationally expensive
SCryptPasswordEncoder	Very High	Low-Medium	High resistance to custom hardware attacks	Memory-intensive, slower than BCrypt
Argon2PasswordEncoder	Very High	Medium	Highly secure, winner of PHC, highly configurable	Requires careful configuration

Update User Service for Password Encoding and Spring Authentication

Key Updates in UserService Class

- ① Implementation of UserDetailsService Interface:
 - Implements UserDetailsService and provides the loadUserByUsername method for integrating with Spring Security.
- ② Password Encoding:
 - Uses PasswordEncoder to encode passwords before saving them.
- ③ Password Verification:
 - Has a commented-out verifyPassword method because Spring Security handles password verification.
 - Previously: Includes a verifyPassword method that compares plain text passwords.
- ④ Authorities and Roles:
 - In loadUserByUsername, assigns default roles (ROLE_USER, ROLE_ADMIN) to the authenticated user.
 - Previously: Did not handle roles or authorities since it does not integrate with Spring Security.
- ⑤ Security Integration:
 - Fully integrated with Spring Security, using UserDetailsService to load user details for authentication.

```
package com.shadsluiter.ordersapp.service;

import com.shadsluiter.ordersapp.data.UserRepository;
import com.shadsluiter.ordersapp.models.UserEntity;
import com.shadsluiter.ordersapp.models.UserModel;

import java.util.ArrayList;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class UserService implements UserDetailsService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Autowired
    public UserService(UserRepository userRepository, PasswordEncoder passwordEncoder ) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    public UserModel save(UserModel userModel) {
        // hash the password before saving
        userModel.setPassword(passwordEncoder.encode(userModel.getPassword()));
        UserEntity userEntity = convertToEntity(userModel);
        UserEntity savedUser = userRepository.save(userEntity);
        return convertToModel(savedUser);
    }

    // will be replaced by LoadUserByUsername in the UserDetailsService using Spring Security
    public UserModel findByLoginName(String loginName) {

        UserEntity userEntity = userRepository.findByLoginName(loginName);
        if (userEntity == null) {
            return null;
        }

        return convertToModel(userEntity);
    }

    // will be replaced when using Spring Security
    public boolean verifyPassword(UserModel user) {
        UserEntity userEntity = userRepository.findByLoginName(user.getUserName());
        if (userEntity == null) {
            return false;
        }
        return passwordEncoder.matches(user.getPassword(), userEntity.getPassword());
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        UserEntity userEntity = userRepository.findByLoginName(username);
        if (userEntity == null) {
            throw new UsernameNotFoundException("User not found");
        }
    }
}
```

```
    }
    List<GrantedAuthority> authorities = new ArrayList<>();
    for (String role : userEntity.getRoles()) {
        authorities.add(new SimpleGrantedAuthority(role));
    }
    return new User(userEntity.getUserName(), userEntity.getPassword(), authorities);
}

public UserModel findById(String id) {
    UserEntity userEntity = userRepository.findById(Long.parseLong(id));
    return convertToModel(userEntity);
}

public void delete(String id) {
    userRepository.deleteById(Long.parseLong(id));
}

public List<UserModel> findAll() {
    List<UserEntity> userEntities = userRepository.findAll();
    List<UserModel> userModels = convertToModels(userEntities);
    return userModels;
}

private List<UserModel> convertToModels(List<UserEntity> userEntities) {
    List<UserModel> userModels = new ArrayList<>();
    for (UserEntity userEntity : userEntities) {
        userModels.add(convertToModel(userEntity));
    }
    return userModels;
}

private UserModel convertToModel(UserEntity userEntity) {
    UserModel userModel = new UserModel();
    userModel.setId(userEntity.getId().toString());
    userModel.setUserName(userEntity.getUserName());
    userModel.setPassword(userEntity.getPassword());
    userModel.setEnabled(userEntity.isEnabled());
    userModel.setAccountNonExpired(userEntity.isAccountNonExpired());
    userModel.setCredentialsNonExpired(userEntity.isCredentialsNonExpired());
    userModel.setAccountNonLocked(userEntity.isAccountNonLocked());
    userModel.setRoles(userEntity.getRoles());
    return userModel;
}

private UserEntity convertToEntity(UserModel userModel) {
    UserEntity userEntity = new UserEntity();
    if (userModel.getId() != null) {
        userEntity.setId(Long.parseLong(userModel.getId()));
    }
    userEntity.setUserName(userModel.getUserName());
    userEntity.setPassword(userModel.getPassword());

    userEntity.setEnabled( userModel.isEnabled());
    userEntity.setAccountNonExpired(userModel.isAccountNonExpired());
    userEntity.setCredentialsNonExpired(userModel.isCredentialsNonExpired());
    userEntity.setAccountNonLocked(userModel.isAccountNonLocked());
    userEntity.setRoles(userModel.getRoles());
    return userEntity;
}
}
```

Update Login Form

Modify the login and registration forms to handle authentication through the Spring Security filter chain instead of the UsersController.

```
<!-- Login.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" xmlns:layout="http://www.ultraq.net"
<head>
    <title>Login</title>
</head>
<body>
    <div layout:fragment="content">
        <h1>Login</h1>
        <form th:action="@{/login}" method="post">
            <label for="username">Login Name:</label>
            <input type="text" id="username" name="username" required/>
            <br/>
            <label for="password">Password:</label>
            <input type="password" id="password" name="password" required/>
            <br/>
            <button type="submit">Login</button>
        </form>
        <div th:if="${error}">
            <p th:text="${error}"></p>
        </div>
    </div>
</body>
</html>
```

Key Updates

- Form Action URL:
 - Original The form action is `@{/users/login}`, indicating that the login form will be submitted to the `/users/login` endpoint which was part of the `UsersController` (now removed in favor of Spring Security).
 - Update: The form action is `@/login`, indicating that the login form will be submitted to the `/login` endpoint, which is part of the Spring Security filter chain.
- Thymeleaf Object Binding:
 - Original: The form uses `th:object="${user}"` to bind the form fields to a user object. This means that the form fields will be bound to the properties of the user object.
 - Update: The form does not use object binding with `th:object`
- Field Names:
 - Original: Uses `th:field="{userName}"` and `th:field="{password}"` to bind the form fields directly to the `userName` and `password` properties of the user object.
 - Update: Uses standard HTML name attributes (`name="username"` and `name="password"`) without Thymeleaf object binding. The `username` and `password` are part of the user object in Spring Security.
- Error Handling:
 - Original: Uses `th:if="error"and th : text =/{error}` to display an error message if there is an error attribute in the model.

- Update: Uses `th:if="{param.error}"` and displays a static error message "Invalid login credentials" if the error parameter is present in the request

Update the User Controller

Ensure the controller uses the new security configurations.

```
// can use spring security instead of this method to authenticate users
@PostMapping("/login")
public String login(@ModelAttribute UserModel user, Model model) {
    UserModel existingUser = userService.findByLoginName(user.getUserName());
    // first check if the user exists
    if (existingUser == null) {
        model.addAttribute("error", "User not found!");
        model.addAttribute("user", user);
        return "login";
    }
    // user exists, check the password
    if (!userService.verifyPassword(user) ){
        model.addAttribute("error", "Invalid password!");
        model.addAttribute("user", user);
        return "login";
    }
    // user exists and password is correct redirect to the home page
    return "redirect:/users/";
}

// show the login form.
@GetMapping("/loginForm")
public String showLoginForm(Model model) {
    model.addAttribute("user", new UserModel());
    model.addAttribute("pageTitle", "Login");
    return "login";
}

@GetMapping("/logout")
public String logout() {
    return "redirect:/users/loginForm";
}
```

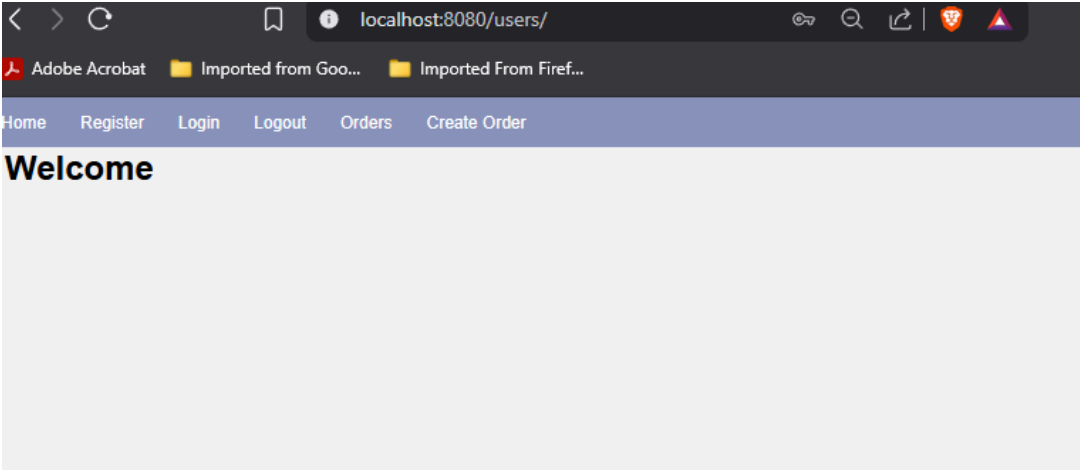
Key Differences

- ① Removed Login Method:
- Original Version: Contains a @PostMapping("/login") method that handles user login by checking user existence and password correctness.
 - New Version: This method is commented out, indicating a shift to using Spring Security's built-in login functionality.
- ② Logout Method Enhancement:
- Original Version: The logout method simply redirects to the login form without any session management.
 - New Version: The logout method now takes an HttpSession parameter and invalidates the session to properly log out the user.
- ③ Imports:
- New Version: Uses jakarta.servlet.http.HttpSession for session management.
 - Original Version: Does not use HttpSession for logout handling.
- ④ The UserController is no longer responsible for processing logins. Spring SecurityConfig relies on the UserService and implements UserDetailsService and the method loadUserByUsername method

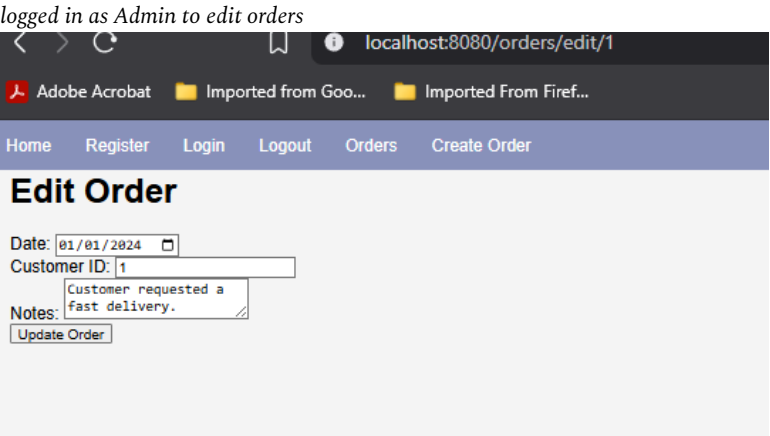
Test the Application*

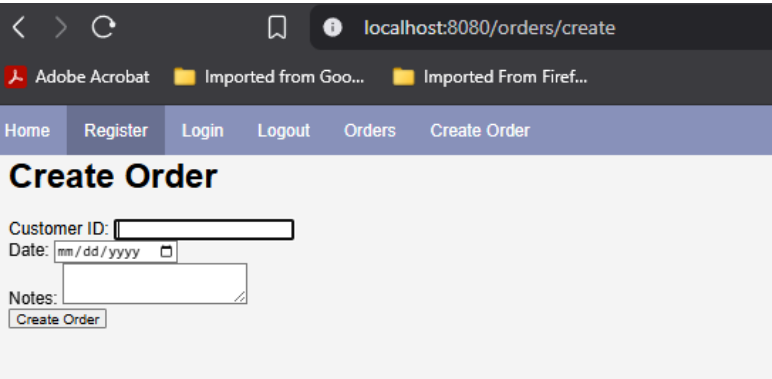
Run the application and ensure that users can register, log in, and access protected resources.

- ① Register a new user.
- ```
2025-09-29T11:57:01.268-07:00 INFO 26792 --- [ordersapp] [nio-8080-exec-3] com.zaxxer.hikar
i.HikariDataSource : HikariPool-1 - Start completed.
2025-09-29T11:57:01.286-07:00 WARN 26792 --- [ordersapp] [nio-8080-exec-3] o.s.s.c.bcrypt.B
CryptPasswordEncoder : Encoded password does not look like BCrypt
2025-09-29T11:57:41.400-07:00 WARN 26792 --- [ordersapp] [nio-8080-exec-1] o.s.s.c.bcrypt.B
CryptPasswordEncoder : Encoded password does not look like BCrypt
2025-09-29T11:57:53.227-07:00 INFO 26792 --- [ordersapp] [nio-8080-exec-6] c.s.o.controller
s.UsersController : User registered: alice1
2025-09-29T11:58:24.947-07:00 INFO 26792 --- [ordersapp] [nio-8080-exec-6] c.s.o.controller
s.UsersController : User registered: alice2
```

- ② Log in with the new user.
- 

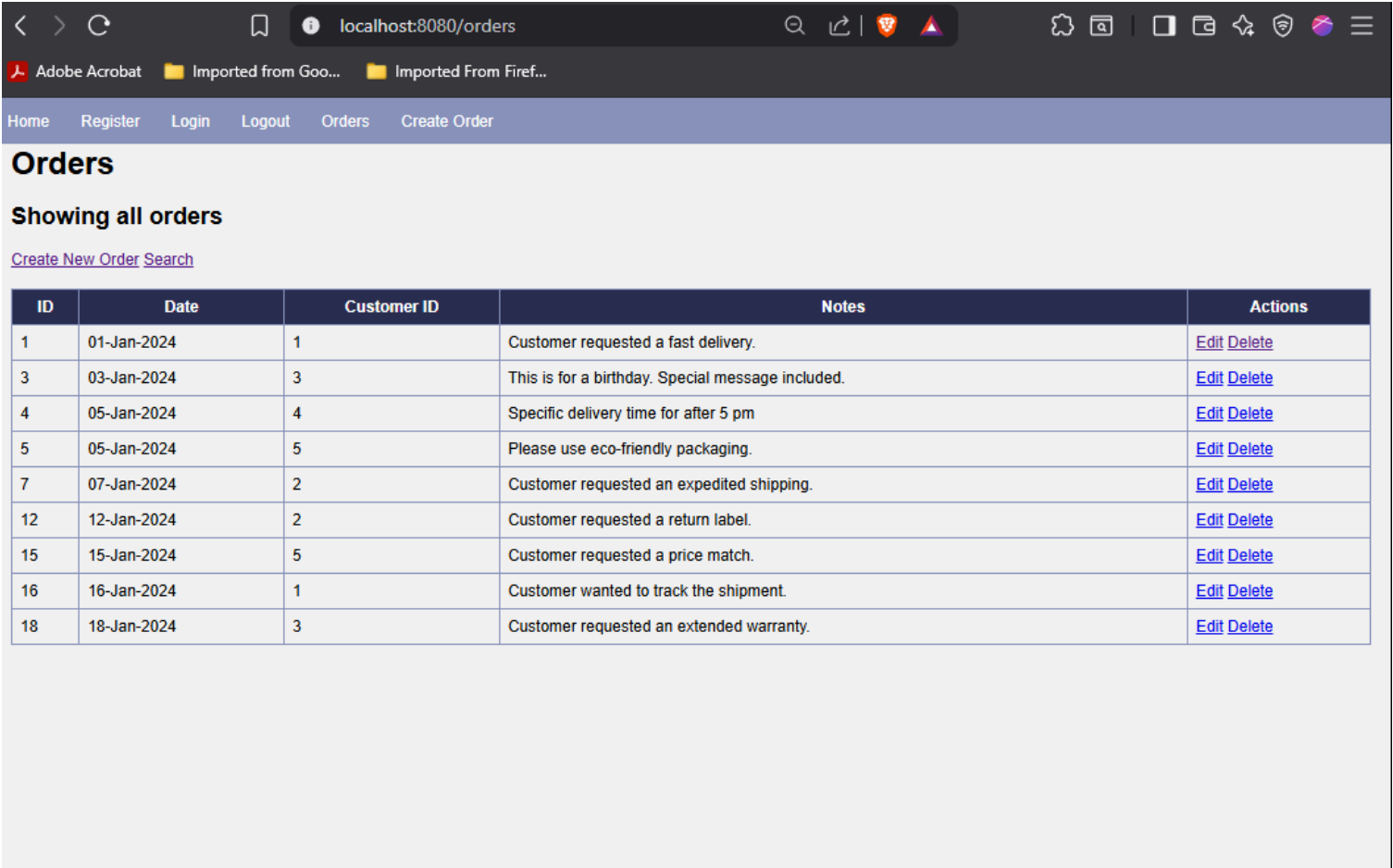
- ③ Access protected resources (e.g., /orders, create, edit).





logged in as Admin to create an order

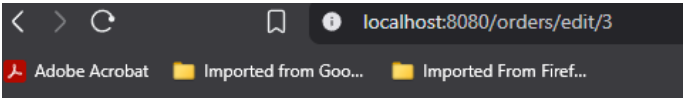
logged in as user to view orders



4. Logout

- ① Attempt to access protected resources.

while logged in with user not admin



## Whitelabel Error Page

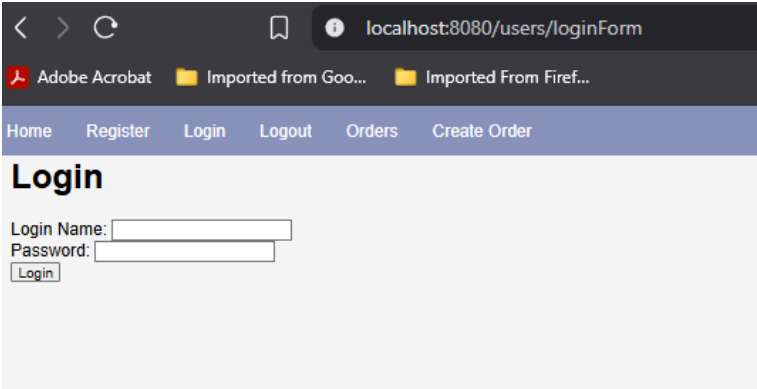
This application has no explicit mapping for /error, so you are seeing this as a fallback.

Mon Sep 29 12:02:37 MST 2025

There was an unexpected error (type=Forbidden, status=403).

Forbidden

while logged out it reroutes to login to perform action



## Part 2 Thymeleaf Security

What can you do with Thymeleaf and Security?

- ① Authorization Tags are used to conditionally display parts of a template based on the current user's roles or authorities.

```
<div sec:authorize="hasRole('ADMIN')">

 This content is only visible to users with the ADMIN role.

</div>
```

- ① Authentication Tags are used to display information about the authenticated user.

```
<p>Welcome, !</p>
```

- ① Access Control Expressions support various expressions for access control checks, such as isAuthenticated(), isAnonymous(), hasRole(), hasAuthority(), etc.

```
<li sec:authorize="isAuthenticated()">

 <a th:href="@{/logout}">Logout


```

- ① Conditional Rendering allows the app to display content based on the logged in user.
- sec:isAuthenticated() renders content only if the user is authenticated.
  - sec:isAnonymous() renders content only if the user is not authenticated.
  - sec:hasRole('ROLE\_USER') renders content only if the user has a specific role.
  - sec:hasAnyRole('ROLE\_USER', 'ROLE\_ADMIN')': renders content if the user has any of the specified roles.
- ② Display User Details allow you to show properties of the authenticated user, such as username and authorities (roles).

```
<p>User: </p>

<p>Authorities: </p>
```

### Update the Application to user Thymeleaf Security Extras

Update pom.xml to include Thymeleaf extras for Spring Security.

```
<!-- Thymeleaf Extras for Spring Security -->
<dependency>
 <groupId>org.thymeleaf.extras</groupId>
 <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

updated thymeleaf security dependency allows us to see if user is logged in on the nav bar



\*This also allows us to see the users role and username when logged in

Summary of Key Concepts

Spring Security integrates authentication (who you are) and authorization (what you can do) using components such as the AuthenticationManager, UserDetailsService, and GrantedAuthority. Passwords must be stored as secure hashes like BCrypt; encoding on save and verifying with matches on login prevents plaintext storage and replay attacks. Modern configuration relies on a SecurityFilterChain bean configured via HttpSecurity rather than the legacy WebSecurityConfigurerAdapter, enabling clear authorization rules (for example, hasRole('ADMIN')) while the security filter chain, not controllers, manages the login and logout flow. With Thymeleaf Extras for Spring Security (thymeleaf-extras-springsecurity6), the UI can react to authentication state—showing content conditionally for isAuthenticated() or hasRole(...)—and display details of the authenticated principal. For database-backed users, a working datasource is essential; authentication will fail if the application cannot reach MySQL. From a UX perspective, good patterns include redirecting anonymous users to the login page, displaying the current username and roles, and hiding administrative options from users who lack the necessary privileges.